

Desktop Pet — 桌面宠物技术文档

符玄桌宠项目

2026 年 7 月 3 日

摘要

本文档为桌面宠物项目的完整技术文档。项目基于 Unity 2022.3 LTS 引擎，使用 Live2D Cubism SDK 5-r.4 渲染《崩坏：星穹铁道》角色「符玄」，通过 DWM 玻璃层透明窗口 (DwmExtendFrameIntoClientArea) 实现桌面宠物效果。文档涵盖系统架构、各模块设计、动画系统、状态机及交互机制。

目录

1	项目概述	5
1.1	项目背景	5
1.2	技术栈	5
1.3	项目状态	5
2	系统架构	6
2.1	整体架构	6
2.2	组件依赖关系	8
3	窗口管理 (WindowOverlay)	8
3.1	DWM 玻璃层透明原理	8
3.2	点击穿透 (Click-Through)	9
3.3	窗口样式	9
4	底部输入栏 (BottomInputBar)	9
4.1	概述	9
4.2	坐标系统	9
4.3	UI 设计	10
4.4	交互逻辑	10
5	物理引擎 (DesktopPet)	10
5.1	坐标系	10
5.2	物理参数	11
5.3	地面任务状态机	11
5.4	暂停/恢复机制	11

6	拖拽挣扎动画 (Live2DRenderer)	12
6.1	设计目标	12
6.2	核心机制	12
6.3	直接物理驱动	12
7	时间/天气响应系统 (TimeWeatherController)	12
7.1	概述	12
7.2	时间系统	13
7.3	天气系统	13
7.4	表情联动	13
7.5	动作权重调节	13
7.6	待机气泡联动	14
8	Live2D 渲染器 (Live2DRenderer)	14
8.1	概述	14
8.2	生命周期	14
8.3	参数驱动系统	14
8.4	空闲动作系统	15
8.4.1	睡眠唤醒恢复	16
8.5	走路动画	16
8.5.1	执行顺序与 Cubism Physics 协作	17
8.6	体态过渡	18
8.7	平滑转身机制	18
8.8	强制动作锁定机制	19
9	交互系统 (DragHandler)	19
9.1	左键拖拽	19
9.2	分区点击反馈	19
9.3	右键菜单	20
9.4	点击穿透管理	20
10	上下文菜单 (ContextMenu)	20
10.1	UI 实现	20
10.2	功能区域	20
10.3	聊天标签设计	21
10.4	动作执行流程	21
11	混合渲染 (HybridRenderer)	21
11.1	设计目标	21
11.2	切换规则	21
11.3	接口设计	22

12 服务端轮询与数据查询 (ServerPollService)	22
12.1 概述	22
12.2 API 端点	22
12.3 轮询机制	23
12.4 与 AI 集成	23
12.4.1 Everything 文件搜索集成	23
12.4.2 协程异步执行架构	24
13 性能监控 (PerformanceMonitor)	25
13.1 概述	25
13.2 监控指标	26
13.3 自适应降频	26
13.4 调试面板	26
14 气泡优先级系统 (ChatBubble)	26
14.1 设计动机	26
14.2 优先级级别	26
14.3 优先级规则	27
14.4 气泡古风装饰	27
14.4.1 紫色云纹角饰	27
14.4.2 闪烁星点系统	27
14.4.3 紫色装饰线	27
15 动画详述	28
15.1 空闲动作详解	28
15.1.1 动作 1 — 歪头	28
15.1.2 动作 2 — 微笑	28
15.1.3 动作 3 — 挑眉	28
15.1.4 动作 4 — 星辉环绕（已移除视觉特效）	28
15.1.5 动作 5 — 伸懒腰	28
15.1.6 动作 6 — 爱心眼	28
15.1.7 动作 7 — 数钱	29
15.1.8 动作 8 — 委屈	29
15.1.9 动作 9 — 法阵显现（已移除视觉特效）	29
15.1.10 动作 10 — 害羞	29
15.1.11 动作 11 — 困惑（AI 触发专用）	29
15.2 眨眼系统	29
15.3 呼吸与微动	30
16 目录结构	30

目录	4
17 开发指南	31
17.1 调试快捷键	31
17.2 调参指南	31
17.3 构建注意	31
18 待改进项	32
18.1 AI 活动感知与性格优化	32
19 版本历史	35

1 项目概述

1.1 项目背景

原桌面宠物程序基于 Windows GDI+ 分层窗口技术 (`text1.c`)，使用静态 PNG 图片硬切换模拟动画。本项目将其迁移至 Unity 引擎，利用 Live2D Cubism SDK 实现参数化变形动画，获得更流畅的视觉效果和更丰富的交互能力。

1.2 技术栈

- 引擎：Unity 2022.3.62t7 (LTS)
- 角色渲染：Live2D Cubism SDK 5-r.4（参数化变形 + 实时物理模拟）
- 窗口管理：Win32 API (`user32.dll` + `Dwmapi.dll`) — DWM 玻璃层透明、置顶、点击穿透
- 交互方式：左键拖拽/抛掷、右键上下文菜单、底部 AI 聊天输入栏
- 语言：C# 脚本
- 平台：Windows 10/11 (Win64)

1.3 项目状态

功能已完成：

- Live2D 模型加载与渲染（自动 AssetDatabase / Resources 双路径）
- 桌面透明窗口（DWM 玻璃层扩展 `DwmExtendFrameIntoClientArea`，黑色 = 透明）
- 鼠标穿透动态管理（`WS_EX_TRANSPARENT`）
- 像素物理引擎（重力、边界碰撞、抛掷）
- 地面任务状态机（5 种行为加权随机切换）
- 走路动画（侧身转体 + 腿臂交叉摆动 + 上下颠簸 + 体态淡入淡出）
- 11 种空闲动作循环（歪头、微笑、挑眉、星辉、伸懒腰、爱心、数钱、委屈、法阵、害羞、困惑）
- 右键上下文菜单（权重调整 + 强制动作播放）
- 强制动作锁定机制（播放期间不被走路覆盖，播完自动恢复）

- 分区点击反馈（头部/身体/腿部 3 区不同反应）
- 拖拽挣扎动画（双臂交替划水 + 双腿交替 + 身体扭动 + 表情）
- 头发/裙子/法盘直接物理驱动（拖拽速度实时输入 CubismPhysics）
- 时间/天气响应系统（昼夜犯困眼皮 + 天气表情基调 + 待机气泡）
- AI 对话 + 自动闲聊 + 底部输入栏（DeepSeek Function Calling, 28+ 工具, 含协程异步）
- 气泡优先级系统（High/Normal/Low, 防止 AI 回复被问候覆盖）
- 服务端推送轮询（定时查询 Node.js 推送队列）
- 微信小程序数据打通（成绩/课表/考试/学业概览实时查询）
- 性能监控（FPS/CPU/内存 + 自适应降频）
- 开机自启（系统托盘一键设置）
- 调试窗口实时调参
- 3D 模型渲染管线预留（HybridRenderer 双渲染器交叉淡入淡出）
- 睡眠唤醒自动恢复：时间间隙检测 + 延迟 2s 重建 DWM + 恢复物理状态
- DWM 崩溃安全模式：连续崩溃跳过 DWM 玻璃层重建

2 系统架构

2.1 整体架构



10			DragHandler (交互)		
11			--- 左键拖拽/抛掷		
12			--- 右键菜单开关		
13			--- 点击穿透管理 (每帧)		
14					
15			Live2DRenderer (渲染, order=801)		
16			--- CubismModel 参数驱动		
17			--- 眨眼/呼吸/Perlin 噪声微动		
18			--- 空闲动作 1-11 循环		
19			--- 走路动画 (转体/抬腿/颠簸/淡入淡出)		
20			--- 强制动作锁定 (actionLocked)		
21			--- 拖拽挣扎 (双臂划水+双腿交替+速度驱动物理)		
22			--- 时间/天气表情联动		
23					
24			ContextMenu (UI)		
25			--- 设置/动作/聊天 3 标签		
26			--- 权重调整 + 强制动作按钮		
27			--- 聊天标签: 仅输入框+发送按钮		
28			--- OnGUI 直接绘制		
29					
30			WindowOverlay (Win32)		
31			--- DwmExtendFrameIntoClientArea (玻璃层透明)		
32			--- SetWindowLong (穿透/置顶)		
33			--- SetWindowPos (全屏置顶)		
34					

35			BottomInputBar (底部输入栏)	
36			--- Windows 搜索风格 (白底+浅灰输入框+蓝按钮)	
37			--- 固定坐标系 (BAR_LEFT/RIGHT/TOP/BOTTOM)	
38			--- 淡入动画 (0.5s)	
39			--- Enter/按钮发送 + 排队中禁用	
40		+	-----+	
41	+	+	-----+	+

2.2 组件依赖关系

关键交互流程:

渲染循环 DesktopPet.Update → Live2DRenderer.LateUpdate (参数设置)

物理更新 Update → StepPet (重力/速度/位置) → UpdateGroundTask (状态机)

强制动作 ContextMenu.DrawActionButton → DesktopPet.ForceStop →
Live2DRenderer.ForceIdleAction → (锁定) → ResetIdleAction (解锁 + 回调)

菜单交互 DragHandler.Update (Block 0: 菜单穿透管理) → ContextMenu.IsOpen

走路淡入 LateUpdate 检测 isWalking 切换 → 启动 walkFadeInRemaining (0.3s 体态渐入)

3 窗口管理 (WindowOverlay)

3.1 DWM 玻璃层透明原理

透明方案从色键 (Color Key / LWA_COLORKEY) 改为 DWM 玻璃层扩展, 彻底解决了半透明像素透绿的问题:

1. Camera 设置 ClearFlags = SolidColor, Background = (0, 0, 0, 0) 纯黑透明
2. 通过 Win32 API DwmExtendFrameIntoClientArea 将窗口客户区黑色像素设为透明
3. 设置 MARGINS 结构: cxLeftWidth = -1 表示全窗口玻璃层扩展
4. 需在 Project Settings 中关闭 DXGI flip model (Graphics API = D3D11)

5. 构建版自动重试 5 次查找窗口句柄（延迟帧防竞态）

相比色键方案的优势：

- 色键（#00FF00）只精确移除纯绿色，Live2D 模型半透明/抗锯齿像素与绿色混合后 $\neq$ #00FF00 $\rightarrow$ 残留绿边
- DWM 玻璃层让黑色（0,0,0）透明，Live2D 半透明像素是黑色系的，不会产生绿色伪影
- 无需保持 WS_EX_LAYERED，窗口样式更简单

3.2 点击穿透 (Click-Through)

- WS_EX_TRANSPARENT 样式让鼠标事件透过透明区域传到桌面
- 每帧根据鼠标位置动态切换：鼠标在宠物区域内 $\rightarrow$ 移除穿透（接收点击）；鼠标在区域外 $\rightarrow$ 添加穿透（点击不受影响）
- 菜单打开时：菜单区域内可交互，区域外穿透（优先于普通穿透逻辑）

3.3 窗口样式

- 全屏无边框窗口（WS_POPUP）
- 置顶（HWND_TOPMOST）
- 不在任务栏显示（WS_EX_TOOLWINDOW）
- 工具窗口风格

4 底部输入栏 (BottomInputBar)

4.1 概述

BottomInputBar 是底部 AI 聊天输入栏，设计为 Windows 搜索风格：白色圆角背景、浅灰输入框、蓝色发送按钮。使用固定坐标系统（BAR_LEFT/RIGHT/TOP/BOTTOM）精确定位。

4.2 坐标系统

使用四个常量定义位置，可在脚本顶部直接修改：

```
1 private const float BAR_LEFT    = 265f;  
2 private const float BAR_RIGHT   = 635f;  
3 private const float BAR_TOP     = 1528f;  
4 private const float BAR_BOTTOM = 1602f;
```

Listing 1: BottomInputBar 坐标常量

四个属性 (BarLeft/Right/Top/Bottom) 公开供 DragHandler 等外部组件使用。

4.3 UI 设计

- 背景：白色半透明矩形 (RGBA: 1,1,1,0.92)，顶部浅灰阴影线
- 输入框：浅灰背景 (0.85,0.85,0.88,0.6)，深色文字，14px 字号
- 发送按钮：蓝色 (0.2,0.5,0.95)，悬停加深，12px 字号
- 输入框与按钮等高，水平排列，间距 4px
- 淡入动画：打开后 0.5s 渐显 (α 从 0 到 1)

4.4 交互逻辑

- Enter 键发送 (需输入框获得焦点且内容非空)
- 按钮点击发送 (聊天队列忙碌时按钮变灰并显示 图标)
- 发送后清空输入框、释放焦点
- 鼠标悬停检测 (IsMouseOverBar 属性) 供 DragHandler 判断点击穿透

5 物理引擎 (DesktopPet)

5.1 坐标系

采用像素坐标系：左上角为原点 (0, 0)，X 轴向右为正，Y 轴向下为正。物理状态全部用整数像素值表示，避免浮点精度问题。

5.2 物理参数

表 1: 核心物理常量

参数	值
重力加速度	1 px/frame ²
最大下落速度	8 px/frame
最大水平速度	12 px/frame
地面 Y 偏移	-300 px (相对于屏幕底部)
速度系数	0.5

5.3 地面任务状态机

宠物共有 5 种地面行为，通过加权随机选择：

表 2: GroundTask 状态机

任务	权重 (默认)	行为
MoveLeftEdge	1	向左走到屏幕左边缘
MoveRightEdge	1	向右走到屏幕右边缘
MoveLeftTime	1	向左走随机时长 (2-4s)
MoveRightTime	1	向右走随机时长 (2-4s)
StopTime	6	停止随机时长 (6-15s)

状态机特点：

- 到达边缘后自动转向：左边缘 → 右走/停止；右边缘 → 左走/停止
- 定时任务到期 → 重新加权随机选择
- 停止任务期间播放空闲动作循环

5.4 暂停/恢复机制

- isPaused 布尔值：暂停时物理 Update 直接 return
- ForceStop()：将边缘任务转为定时任务，清零 petVx，启动停止任务
- Resume()：恢复时如果当前无任务或停止中，启动新任务
- Pause(float)：暂停指定秒数后自动恢复

6 拖拽挣扎动画 (Live2DRenderer)

6.1 设计目标

拖拽时角色应表现出被“提着走的”挣扎感，同时裙子和头发有惯性拖尾效果。物理系统（CubismPhysicsController）在拖拽中继续运作，但参数值会被挣扎动画覆盖。

6.2 核心机制

拖拽挣扎动画通过帧间速度驱动：每帧计算 $\Delta x = petX_t - petX_{t-1}$ ，经平滑滤波后驱动各参数：

- 双臂交替划水：右臂正 = 向前，左臂负 = 向前，同相位实现交替
 - 大幅摆动：Param94（上臂旋转，范围 [-30,60]）
 - 小幅：Param31/32/33（右臂），Param34/36/37（左臂）
 - 手部透视图层跟随幅度（自动浮到衣服前）
- 双腿交替：Param126/129 腿前后摆动，Param127/131 腿弯曲
- 身体扭动：ParamBodyAngleY 跟随鼠标方向平滑转身
- 表情：瞪眼 + 张嘴（慌“张表”情）

6.3 直接物理驱动

帧间速度经平滑后直接写入 Cubism 参数，为 CubismPhysics 提供输入信号：

- 裙子：Param82（前摆动）、Param87（后摆动）、Param84（侧摆动）
- 法盘：Param49/51/57/60（法盘旋转/摆动）
- 头发：20 个输出参数（Param5/7/9/11/14/17/19/21/23/35/41/43/45/55/62/91/74/89/169 等）
- 头部：ParamAngleX/Z 跟随速度方向

所有直接驱动参数绕过物理系统的 Delay=0.8s 延迟，实现即时响应。

7 时间/天气响应系统 (TimeWeatherController)

7.1 概述

TimeWeatherController 负责感知系统时间和当地天气，向 Live2DRenderer 提供状态信息，使宠物的表情、动作权重和待机气泡内容随昼夜/天气变化。

7.2 时间系统

- 使用 `System.DateTime.Now` 获取当地小时 (0-23)
- `isNight`: 18:00 - 6:00 (夜间模式)
- `isSleepyTime`: 22:00 - 5:00 (犯困时段)
- `dayPhase`: 6:00 = 0.0, 12:00 = 0.5, 18:00 = 1.0 (用于渐变)

7.3 天气系统

- 通过 `wttr.in` API 获取当地天气 (城市可配, 留空 = 自动 IP 定位)
- 轮询间隔默认 300 秒 (可配)
- 解析天气类型: Clear / Cloudy / Overcast / Rain / Drizzle / Thunder / Snow / Fog
- 同时解析温度 (用于气泡文本)
- 首次启动立即获取, 失败时保留 Unknown 状态

7.4 表情联动

在 `UpdateIdleAnimation()` 中叠加昼夜/天气基调表情:

表 3: 昼夜/天气表情联动

条件	表情
犯困时段 (22-5 点)	眼皮微垂 (EyeOpen lerp 0.7)
夜晚 (18-22 点)	轻度眼皮下垂
阴雨/雷雨/阴天	眉毛微抬 + 嘴巴微嘟 (委屈)
晴/多云	自然微笑
雪	微微张嘴 (好奇) + 眼睛睁大

7.5 动作权重调节

`PickWeightedIdleAction()` 动态调整权重:

- 夜间: 星辉/爱心眼减少, 委屈增加
- 犯困时段: 委屈 +2, 歪头 +1
- 雨天: 委屈 +2, 爱心眼/数钱减少

- 晴天：爱心眼 +1，星辉 +1
- 雪天：挑眉 +1，委屈 +1

7.6 待机气泡联动

PickIdleBubbleMessage() 50% 概率触发时间/天气特化气泡：

- 犯困时段：” 好晚了…该睡了 ”” 眼睛快睁不开了…”
- 夜晚：” 晚上好呀 ”” 月色真美…”
- 早晨（5-8 点）：” 早安 ”” 今天也要加油哦！ ”
- 雨天：” 下雨了呢 ”” 听雨声发呆 ”
- 雷雨：” 打雷了好可怕！ ”” 轰隆隆…好吓人…”
- 雪：” 下雪了！好漂亮 ”” 想堆雪人…”
- 晴天：” 今天天气真好 ”” 太阳暖洋洋的 ”

8 Live2D 渲染器 (Live2DRenderer)

8.1 概述

Live2DRenderer 是核心渲染组件，通过 Cubism SDK 驱动模型参数实现所有动画。继承自 IPetRenderer 接口，被 HybridRenderer 包装管理。

8.2 生命周期

1. Start: 自动加载模型（优先 AssetDatabase，降级 Resources.Load）
2. Update: 计算走路相位 + 垂直颠簸偏移 + 模型位置同步
3. LateUpdate: 设置所有参数（眨眼、呼吸、噪声、空闲动作/走路）

8.3 参数驱动系统

使用 SetParameter(name, value) 统一设置 Cubism 参数，涵盖约 60+ 个参数 ID：

- 身体：ParamBodyAngleX/Y/Z、ParamBreath
- 头部：ParamAngleX/Y/Z、ParamEyeBallX/Y
- 眼睛：ParamEyeLOpen/ROpen、ParamEyeLSmile/RSmile

- 嘴部: ParamMouthForm
- 眉毛: ParamBrowRY/LY
- 特效: Param121 (黑幕切换)、Param132 (眼镜发光)、Param431/911/741 (星辉环显隐) 等

8.4 空闲动作系统

共有 11 种空闲动作循环播放 ($1 \rightarrow 2 \rightarrow \dots \rightarrow 11 \rightarrow 1$):

表 4: 空闲动作列表

ID	名称	时长 (s)
1	歪头 (Tilt)	2.0
2	微笑 (Smile)	2.0
3	挑眉 (Brow)	2.0
4	星辉 (StarSpin)	6.0
5	伸懒腰 (Stretch)	4.5
6	爱心 (HeartEyes)	3.0
7	数钱 (Money)	3.5
8	委屈 (Cry)	3.5
9	法阵 (MagicCircle)	5.5
10	害羞 (Blush)	3.5
11	困惑 (Confused)	3.5

每个动作使用缓入缓出曲线 $f(t) = \sin(t\pi)$, 其中 $t \in [0, 1]$ 。动作结束时调用 `ResetIdleAction()` 清理参数。

选择策略: 每次播放完, 通过加权随机选取下一个。各动作权重可调:

表 5: 空闲动作权重

动作	权重	相对概率
歪头	5	16.1%
微笑	5	16.1%
挑眉	3	9.7%
星辉	2	6.5%
伸懒腰	2	6.5%
爱心	3	9.7%
数钱	2	6.5%
委屈	3	9.7%
法阵	1	3.2%
害羞	3	9.7%
困惑	0	0.0%

暂停时: `isPaused = true` 时不播新动作（菜单打开时冻结动作循环）。

8.4.1 睡眠唤醒恢复

Windows 睡眠/唤醒后,Unity 的 `OnApplicationPause` 在某些系统上不可靠 (Deepin/统信 UOS 不触发, Windows 特定环境偶发不触发)。WindowOverlay 通过帧间时间间隙检测 (`Time.realtimeSinceStartup` 差值) 来补偿唤醒恢复:

- `OnResumeFromSleep()` 被 `DesktopPet.Update()` 在检测到时间间隙后调用
- 保持 `_suspended = true` 禁止所有 Win32 调用 (`SetWindowLong/SetWindowPos`)
- 延迟 2 秒重建窗口设置, 等待 DWM/GPU 驱动完全恢复
- 连续崩溃安全模式: `PlayerPrefs._skip_dwm_rebuild` 跳过 DWM 玻璃层重建
- 重建完成后同步恢复: `pet.isPaused = false, pet.onGround = false` (强制重新找地面)

8.5 走路动画

采用横版侧面走路风格, 所有参数在 `LateUpdate` 中设置, 利用 Cubism SDK 的 `ICubismUpdatable` 执行顺序机制确保物理与动画正确协作:

- 转体: `ParamBodyAngleY = 18°` (身体转向侧面, 方向自动匹配翻转)
- 前倾: `ParamBodyAngleX = 5°`

- 低头: $\text{ParamAngleY} = 8^\circ$
- 步频: 5 Hz (WALK_SWAY_FREQ)
- 抬腿: Param165 (左) / Param164 (右), 幅度 4
- 摆臂: 交叉对位 (左腿前 $\rightarrow$ 右手前), 分大小范围参数:
 - 大范围: Param94 (右手上臂旋转, 范围 $[-30, 60]$), $\text{legPhase} \times 2$
 - 小范围: Param31/32/33 (右臂, 范围 $[-1, 1]$), $\text{legPhase} \times 0.4$
 - 左臂: Param34/36/37 (左臂, 范围 $[-1, 1]$), $\text{rightPhase} \times 0.4$
- 颠簸: $1 - |\sin(\text{phase})| \times 4 \text{ px}$ (迈步时最低)
- 暂停时归零: `isPaused = true` 时 `_walkPhase` 和 `_walkBounceOffset` 立即清零, 防止睡眠唤醒后原地上下颠簸
- isPhysActive 守卫: 新增 `isPhysActive = (_pet != null && !_pet.isPaused)` 布尔判断, 仅在物理活跃且在地面且有水平速度时累积走路相位
- 呼吸加深: `ParamBreath = 3 + 相位波`

8.5.1 执行顺序与 Cubism Physics 协作

Cubism SDK 通过 `CubismUpdateController` 统一调度所有 `ICubismUpdatable` 组件, 各组件按 `ExecutionOrder` 顺序执行:

表 6: Cubism 更新执行顺序

组件	ExecutionOrder	功能
<code>Live2DRenderer.Update</code>	Unity Update	设体态参数
<code>CubismPhysicsController</code>	800	物理模拟驱动衣服飘动
<code>Live2DRenderer.LateUpdate</code>	801	设腿/臂摆动 + <code>ForceUpdateNow</code>
<code>CubismRenderController</code>	10000	面向相机

关键设计: 体态参数 (身体转体/前倾/低头)

在 `Update()` 中提前设置, 使 `CubismPhysicsController(800)`

在 `LateUpdate` 阶段能读取到正确的走路体态来计算衣服飘动。

手臂参数在 `LateUpdate(801)` 中设置 (晚于物理),

再通过 `ForceUpdateNow()` 强制网格重算,

确保手臂摆动值覆盖物理输出, 同时衣服飘动由物理驱动保持自然。

常见问题:

1. 不加 [DefaultExecutionOrder(801)] 时, LateUpdate 与 Cubism 更新顺序不确定, 可能导致物理覆盖手臂参数 (手臂值被物理置零)
2. 不加 ForceUpdateNow() 时, 网格使用 Update 阶段的旧参数计算, 手臂参数虽已设但网格不变 (视觉上不生效)
3. 体态不在 Update 中提前设时, 物理使用空闲体态驱动衣服, 走路时衣服呈空闲态悬挂 (不随身体飘动)

8.6 体态过渡

表 7: 过渡参数

过渡	时长	曲线
走路 → 空闲 (消退)	0.4s	二次缓出
空闲 → 走路 (淡入)	0.3s	二次缓入

走路 → 空闲时, 走路的体态参数 (转体/前倾/低头) 经 0.4s 渐消, 同时空闲参数已经开始播放, 实现平滑过渡。空闲 → 走路时, 腿臂立刻摆动 (走路感瞬间有), 体态经 0.3s 渐入 (不走硬边)。

8.7 平滑转身机制

方向切换 ($petVx$ 变号) 时, 模型 `scale.x` 不再瞬间取反, 改为插值渐变:

- 新增字段 `_smoothScaleX` 存储当前朝向 (+1 = 右, -1 = 左)
- 每帧计算目标符号 `targetSign`:
 - 拖拽中: 保持当前 `_smoothScaleX` 符号
 - $petVx > 0$: $targetSign = +1$
 - $petVx < 0$: $targetSign = -1$
 - $petVx = 0$: 保持最后朝向 (不恢复默认右)
- `_smoothScaleX = Mathf.Lerp(_smoothScaleX, targetSign, Time.deltaTime × TURN_SPEED)`
- `TURN_SPEED = 10`, 约 0.15s 完成完整转身
- 差值 < 0.005 时直接赋精确值, 消除残差抖动

8.8 强制动作锁定机制

右键菜单触发强制动作时的保护机制：

1. ForceIdleAction(id) 设置 `_actionLocked = true`
2. isWalking 检测条件中加入 `! _actionLocked`，锁定期间不切走路
3. 动作播放完毕 → ResetIdleAction() 解锁 + 触发 OnForcedActionFinished 事件
4. 回调恢复宠物运动（取消暂停 + Resume）

9 交互系统 (DragHandler)

9.1 左键拖拽

1. 鼠标在宠物区域内按下左键 → 记录拖拽起点
2. 移动超过 5px 阈值进入拖拽模式
3. 拖拽中：SetWindowPos 跟随鼠标移动，关闭穿透
4. 释放时：计算末速度（缓冲滑动平均），施加给宠物（保证 $v_y > 2$ ）
5. 进入自由落体状态

9.2 分区点击反馈

点击时根据点击位置（归一化高度 0-1）分 3 区反馈：

- 头部区域 ($\text{hitNormY} < 0.35$)：戳脸一睁大眼 + 张嘴 + 挑眉，如” 呜哇！戳到脸了”
- 身体区域 ($\text{hitNormY} < 0.65$)：摸身体一害羞低头 + 微笑 + 眼神躲闪
- 腿部区域 ($\text{hitNormY} \geq 0.65$)：踢腿一腿向前伸 + 坏笑 + 眯眼

反应持续期间设置 `_poseLocked = true`，参数保存在 `_clickSavedParams` 字典中，LateUpdate 重新写入以对抗 CubismPhysicsController 的覆盖。

9.3 右键菜单

1. 鼠标在宠物区域内按下右键 → 打开 ContextMenu
2. 菜单打开时：区域内可交互，区域外点击穿透
3. 右键再次点击可关闭菜单
4. 点击动作按钮：ForceStop → ForceIdleAction → 菜单关 UI → 动作播完自动恢复

9.4 点击穿透管理

每帧执行 (DragHandler.Update):

1. 菜单打开时：优先菜单穿透规则
2. 菜单关闭时：鼠标在宠物矩形内 → 关穿透；鼠标在外 → 开穿透
3. 确保猫爪 (WS_EX_TRANSPARENT) 不吞掉输入事件

10 上下文菜单 (ContextMenu)

10.1 UI 实现

使用 Unity 的 OnGUI 系统直接绘制 (无需 Canvas / UI Prefab)，适用于透明窗口架构。

样式特点：

- 深色 VS Code 风格背景 (0.15, 0.15, 0.17, 0.95)
- 粉色标题 (0.9, 0.6, 0.8) + 粉色强调色 (0.8, 0.4, 0.6)
- 半透明圆角矩形背景
- 可滚动内容区域

10.2 功能区域

菜单分为三个标签页 (Settings / Actions / Chat)，通过 GUI.Toolbar 切换。

1. 设置 (Settings): 5 种地面任务的权重滑块 (0-10)，应用按钮即时生效
2. 动作 (Actions): 11 个动作按钮，点击后强制播放对应动作
3. 聊天 (Chat): 简化界面—仅底部输入框 + 发送按钮，无消息历史/状态栏/工具栏

10.3 聊天标签设计

聊天标签 (DrawChatTab) 已大幅简化:

- 使用 `GUILayout.FlexibleSpace()` + `GUILayout.BeginVertical(MinHeight)` 将输入区域推至标签底部
- 移除了消息历史区域、错误/状态提示、工具栏
- 输入框 + 发送按钮与 `BottomInputBar` 同时可用
- 发送逻辑: 调用 `ChatManager.SendMessage()`, 队列忙时按钮禁用

10.4 动作执行流程

点击动作按钮时的完整流程:

1. `ForceStop()` → 停止走路, 清零 `petVx`, 切到 `StopTime`
2. `ForceIdleAction(id)` → 设置 `_actionLocked = true` + 播放动作
3. `_isOpen = false` → 菜单 UI 关闭 (`OnGUI` 不再绘制)
4. 注册 `OnForcedActionFinished` 回调
5. 动作自然播完 (easing 曲线结束) → `ResetIdleAction()` 触发回调
6. 回调: 取消暂停 + `Resume` → 宠物自动开始走路/停止
7. 走路体态经 0.3s 淡入, 过渡平滑

11 混合渲染 (HybridRenderer)

11.1 设计目标

为未来引入 3D 模型渲染预留的混合渲染层。当前仅 Live2D 渲染器活跃, 3D 渲染器已挂载但 disabled。

11.2 切换规则

- 空闲/点击/停止 → Live2D (精细表情 + 物理飘动)
- 走路/飞行/空中 → 3D (骨骼动画, future work)
- 过渡方式: Alpha 交叉淡入淡出 (0.3s)

11.3 接口设计

IPetRenderer 接口定义了通用渲染器契约:

```
1 public interface IPetRenderer
2 {
3     void ShowDragPose();
4     void ShowClickPose();
5     void ShowLandPose();
6     void ShowWalkPose();
7     void ShowStopPose(float lockSeconds);
8     void OnPetUpdate(int petX, int petY, int petWidth, int petHeight,
9                     int petVx, int petVy, bool onGround,
10                    bool isDragging, bool isPaused);
11 }
```

Listing 2: IPetRenderer 接口

12 服务端轮询与数据查询 (ServerPollService)

12.1 概述

ServerPollService 负责与微信课表小程序的后端 Node.js 服务通信, 实现:

- 推送轮询—定期查询服务端推送队列, 消费提醒/通知
- 数据查询—供 AI 模块调用, 实时查询学业数据 (成绩、课表、考试)
- 开机通知—宠物启动时向服务端发送开机事件

12.2 API 端点

通过 HTTP GET 请求 Node.js 服务 (localhost:3000), 使用 DESKTOP_TOKEN 鉴权:

表 8: ServerPollService API 端点

端点	方法	功能
/api/pet/poll	GET	获取推送队列
/api/pet/heartbeat	GET	开机心跳通知
/api/pet/scores	GET	查询全部成绩
/api/pet/schedule	GET (+week=N)	查询课表
/api/pet/exams	GET	查询考试安排
/api/pet/user/status	GET	学业概览

12.3 轮询机制

- 启动 5 秒后开始首次轮询
- 间隔 10 秒自动查询
- 每批最多消费 10 条推送（防积压）
- 网络错误自动重试，不影响主循环

12.4 与 AI 集成

AI 模块通过 ToolCallInvoker 注册了 28+ 个工具术式，涵盖系统操作、文件搜索、学业数据查询和角色控制：

表 9: AI 工具术式一览（部分）

术式名称	分类	用途
open_url	系统	在浏览器中打开网址
open_app	系统	启动应用/打开文件
search	系统	Bing 搜索引擎查询
take_screenshot	系统	截取当前屏幕
set_volume	系统	调节系统音量
search_files	文件	Everything 毫秒级文件搜索（协程异步，回退递归）
list_files	文件	列出目录内容
open_folder	文件	在资源管理器中打开文件夹
set_reminder	便签	设置定时提醒（支持重复）
query_reminders	便签	查询待办提醒
query_scores	学业	成绩查询（协程异步）
query_schedule	学业	课表查询（协程异步）
query_exams	学业	考试查询（协程异步）
query_user_status	学业	学业概览（协程异步）
set_expression	角色	切换面部表情
play_action	角色	播放复合动作动画
get_weather	环境	读取本地天气数据

12.4.1 Everything 文件搜索集成

search_files 术式在 Windows 文件搜索方面实现了重大优化：

- **优先 Everything:** 自动检测并调用 Everything CLI (es.exe)，利用其 USN Journal 索引实现全盘毫秒级搜索

- **自动检测**：启动时在 Program Files、LocalAppData 及 PATH 中搜索 es.exe
- **路径限定**：支持 root 参数限定搜索范围为特定目录
- **智能回退**：若未安装 Everything，自动切换到递归目录搜索（10 秒超时）
- **最多 200 结果**：防止返回过多结果导致消息过长

聊天时 AI 自动判断需要调用的术式，从服务端获取真实数据后以白话回复用户。

12.4.2 协程异步执行架构

5 个涉及网络 IO 或后台线程的工具通过 Unity 协程异步执行，避免阻塞主线程：

```

1 // ChatManager.DoToolLoop() 核心调度逻辑
2 string result;
3 if (toolInvoker && toolInvoker.IsCoroutineTool(call.name))
4 {
5     yield return StartCoroutine(
6         toolInvoker.ExecuteCoroutine(call.name, call.arguments));
7     result = toolInvoker.GetCoroutineResult();
8 }
9 else
10 {
11     result = toolInvoker
12         ? toolInvoker.Execute(call.name, call.arguments, out _)
13         : "法阵未就绪";
14 }

```

Listing 3: ChatManager.DoToolLoop 协程调度

```

1 private void RegisterCoroutineTools()
2 {
3     _coroutineExecutors = new Dictionary<string, Func<string,
4         IEnumerator>>
5     {
6         ["query_exams"] = args => RunAsyncTool(
7             () => FindObjectOfType<ServerPollService>()
8                 ?.QueryUpcomingExamsAsync() ...),
9         ["query_scores"] = args => RunAsyncTool(
10            () => FindObjectOfType<ServerPollService>()
11                ?.QueryScoresAsync() ...),
12         ["query_schedule"] = args => RunAsyncTool(
13            () => FindObjectOfType<ServerPollService>()
14                ?.QueryScheduleAsync(week) ...),

```

```

14         ["query_user_status"] = args => RunAsyncTool(
15             () => FindObjectOfType<ServerPollService>()
16                 ?.QueryUserStatusAsync() ...),
17         ["search_files"] = args => RunAsyncTool(
18             () => SearchFilesTask(args)),
19     };
20 }
21
22 // 将 async Task 包装为每帧检查的非阻塞协程
23 private IEnumerator RunAsyncTool(Func<Task<string>> taskFactory)
24 {
25     var task = taskFactory();
26     while (!task.IsCompleted) { yield return null; }
27     _coroutineResult = task.IsFaulted
28         ? $"出错: {task.Exception?.InnerException?.Message}"
29         : task.Result;
30 }

```

Listing 4: ToolCallInvoker 协程工具注册与 RunAsyncTool

协程异步设计要点:

- 同步执行 (23+ 工具): open_url、take_screenshot、set_volume 等瞬时系统操作
- 协程异步执行 (5 工具): 4 个学业查询 (HTTP 请求) + search_files (后台线程文件搜索, Task.Run)
- RunAsyncTool 将 async Task<string> 包装为 IEnumerator, 每帧检查 task.IsCompleted, 不阻塞 Unity 主线程
- 协程工具通过 IsCoroutineTool() 查询, GetCoroutineResult() 获取结果
- search_files 在 Task.Run 后台线程中运行 Everything CLI 或递归搜索, 完全不阻塞主线程
- DoToolLoop 最多 5 轮工具循环, 每轮可包含多个并行 tool_call

13 性能监控 (PerformanceMonitor)

13.1 概述

PerformanceMonitor 提供对桌面宠物运行性能的实时监控。基于 System.Diagnostics.PerformanceCounter 实现, 不依赖第三方库。

13.2 监控指标

表 10: 性能监控指标

指标	方式	采样间隔
FPS	Time.deltaTime 滑动平均	每帧
CPU 使用率	PerformanceCounter("Processor", "% Processor Time", "_Total")	1 秒
内存使用量	PerformanceCounter("Process", "Working Set")	1 秒

13.3 自适应降频

当 FPS 持续低于阈值(默认 30)时,PerformanceMonitor 可通过事件通知 Live2DRenderer:

- 减少 Perlin 噪声微动频率
- 降低空闲动作切换频次
- 减少待机气泡触发

13.4 调试面板

数据通过 DebugWindow 以 OnGUI 形式实时显示, 包括:

- 当前 FPS (带颜色指示: 绿色 ≥ 55 、黄色 30-55、红色 < 30)
- CPU 使用率百分比
- 工作集内存占用量 (MB)

14 气泡优先级系统 (ChatBubble)

14.1 设计动机

AI 回复过程中, 自动闲聊系统可能因定时器触发而弹出低优先级问候气泡, 覆盖正在显示的 AI 回复内容。优先级系统确保重要消息不被中断。

14.2 优先级级别

表 11: 消息优先级

级别	数值	适用场景
Low	0	闲话、定时问候、待机气泡
Normal	1	提醒、交互回应 (戳/摸)
High	2	AI 主动回复

14.3 优先级规则

- 高优先级消息显示期间，低优先级消息被忽略
- 同优先级消息总是可以覆盖当前消息
- `ShowMessage()` 默认使用 Normal 优先级
- `ShowMessage(text, duration, High)` 用于 AI 回复
- 外部可通过 `IsShowingHighPriority` 属性查询当前是否为高优先级展示

14.4 气泡古风装饰

气泡在功能之外增加了符玄主题的视觉装饰，全部代码生成，无需额外贴图资源：

14.4.1 紫色云纹角饰

- 左上角 + 右下角各一朵紫色卷草云纹
- 使用极坐标螺旋算法： $\text{val} = \sin(\theta \times 3 + d \times 10)$
- 从角落沿对角线渐变，加粗线条 + 高对比度
- 角饰大小 28px，透明度 60%，颜色 (0.72, 0.48, 0.84)

14.4.2 闪烁星点系统

- 12 颗独立星星散布在气泡表面，每颗拥有不同的大小、闪烁速度和相位
- 每帧实时呼吸动画： $\text{pulse} = \sin^2(t \cdot \text{speed} + \text{phase})$ ，谷值接近熄灭，峰值全亮
- 每颗星使用高斯光晕纹理渲染 (e^{-3r^2} 衰减)
- 尺寸 5–9px，颜色浅紫 (0.85, 0.65, 0.95)，满透明度
- 闪烁速度 1.2–3.5 Hz 随机分布，产生此起彼伏的星空效果

14.4.3 紫色装饰线

- 气泡顶部一条 3px 高的水平紫色线条（同 `accentColor`）
- 距顶部 2px，左右留白 5px，透明度 75%

15 动画详述

15.1 空闲动作详解

15.1.1 动作 1 — 歪头

- 使用 $\sin(t\pi/2)$ 缓入缓出曲线
- $\text{ParamAngleZ} = 8^\circ$ （歪头）、2s 完成

15.1.2 动作 2 — 微笑

- $\text{ParamEyeLSmile} / \text{RSmile} = 0.6$ （眯眼笑）
- $\text{ParamMouthForm} = 0.4$ （张嘴微笑）
- 2s 完成

15.1.3 动作 3 — 挑眉

- $\text{ParamBrowRY} / \text{LY} = 6^\circ$ （眉毛抬起）
- 2s 完成

15.1.4 动作 4 — 星辉环绕（已移除视觉特效）

已移除所有视觉特效参数（紫环旋转、黑幕、眼镜发光、星星闪烁），仅保留动作时长（6s）和空壳函数。过渡到 DWM 透明方案后，半透明特效像素与黑色背景不兼容。

15.1.5 动作 5 — 伸懒腰

- 右手切换 + 伸出（ $\text{Param92} + \text{Param118} + \text{手指张合}$ ）
- 身体后仰（ $\text{ParamBodyAngleX} = -5^\circ$ ）
- 眯眼 + 张嘴（打哈欠表情）+ 深吸一口气
- 4.5s 梯形曲线（快起 → 保持 → 快落）

15.1.6 动作 6 — 爱心眼

- 爱心眼闪烁（ $\text{Param109} = \text{爱心眼开关}$ ）
- 歪头 12° + 微笑 + 身体微倾
- 3s 完成

15.1.7 动作 7 — 数钱

- 钱表情 (Param122) + 双眼放光 (Param121/137/132)
- 美滋滋微笑 + 歪头 + 身体摇晃
- 3.5s 完成

15.1.8 动作 8 — 委屈

- 泪眼 (Param130) + 低头 (ParamAngleY = 6°)
- 嘴巴微颤 + 八字眉 (眉头抬起)
- 身体微微抽动 (抽泣感)
- 3.5s 完成

15.1.9 动作 9 — 法阵显现 (已移除视觉特效)

已移除所有视觉特效参数 (黑幕、白圈、七星盘、眼镜发光、镜头缩放), 仅保留五阶段手势变化: 举过头顶 → 剑指成型 → 指尖凝光 → 扩散 → 消散。身体姿态和剑指手势仍保留, 视觉光圈参数全部移除。

15.1.10 动作 10 — 害羞

- 黑脸 (Param101) + 低头 + 眼神躲闪
- 害羞微笑 + 眯眼 + 身体扭捏
- 3.5s 完成

15.1.11 动作 11 — 困惑 (AI 触发专用)

- 歪头 + 皱眉 + 眯眼
- 权重为 0 (不自发触发, 仅外部强制调用)
- 3s 完成

15.2 眨眼系统

- 间隔随机 2-5s 一次
- 使用 $|\sin(20t)|$ 快速闭眼睁眼 (0.15s)
- 通过 ParamEyeLOpen / ROpen 控制

15.3 呼吸与微动

使用 Perlin 噪声实现自然微动:

- 呼吸: $\text{ParamBreath} = \text{PerlinNoise}(\text{phase}, 0) \times 0.4$
- 身体晃动: 3 轴 Perlin 噪声 $\times$ 幅度常量
- 头部微动: 2 轴 Perlin 噪声 $\times 0.6/0.4$
- 眼球浮动: 2 轴 Perlin 噪声 $\times 3/2$

16 目录结构

```

1 Desktop_per_pro/
2 +-- README.md # 项目说明
3 +-- project_brief/
4 |   +-- report.pdf # 本文档 (PDF)
5 |   +-- report.tex # 本文档 (LaTeX)
6 +-- code/desktop_unity/
7     +-- desktop_unity.sln # Visual Studio 解决方案
8     +-- Assets/
9         |   +-- Scenes/
10        |   |   +-- scene.scene # 主场景
11        |   +-- Scripts/
12        |   |   +-- DesktopPet.cs # 主控 + 物理 + 状态机
13        |   |   +-- Live2DRenderer.cs # Live2D 渲染 + 动画 + 物
14        |   |   |   理 驱 动
15        |   |   +-- ContextMenu.cs # 右键菜单 UI
16        |   |   +-- DragHandler.cs # 拖拽/抛掷/点击交互
17        |   |   +-- WindowOverlay.cs # Win32 透明窗口
18        |   |   +-- TimeWeatherController.cs # 昼夜/天气响应
19        |   |   +-- ChatBubble.cs # 头顶气泡 (含优先级系统)
20        |   |   +-- ChatManager.cs # AI 对话管理 + Function
21        |   |   |   Calling + 协程调度
22        |   |   +-- AutoChat.cs # 自动闲聊
23        |   |   +-- BottomInputBar.cs # 底部输入栏
24        |   |   +-- ReminderManager.cs # 便签提醒 + Server 酱3 推
25        |   |   |   送
26        |   |   +-- ServerPollService.cs # 服务端轮询 + 数据查询
27        |   |   +-- PerformanceMonitor.cs # FPS/CPU/内存监控
28        |   |   +-- SystemTrayManager.cs # 系统托盘图标
29        |   |   +-- DebugWindow.cs # 调试调参面板

```

```

27 | | +-- ToolCallInvoker.cs # AI 工具调用 (28+ 工具,
    | | 含协程异步)
28 | | +-- PetConfig.cs # 天机簿 — 配置持久化
29 | | +-- PetMemory.cs # 忆境 — 长期记忆系统
30 | | +-- HybridRenderrer.cs # 混合渲染管理器
31 | | +-- IPetRenderrer.cs # 渲染器接口
32 | | +-- Model3DRenderrer.cs # 3D 渲染器 (预留)
33 | +-- Live2D/
34 | | +-- Models/Fuxuan/ # 符玄 Live2D 模型
35 | +-- Plugins/ # Cubism SDK
36 +-- Packages/
37 | +-- manifest.json # Unity 包管理
38 +-- ProjectSettings/ # Unity 项目设置

```

17 开发指南

17.1 调试快捷键

- 1: 强制切换到 Live2D 渲染 (HybridRenderrer 调试用)
- 2: 强制切换到 3D 渲染 (HybridRenderrer 调试用)
- 控制台日志: 各组件关键事件均有 Debug.Log 输出

17.2 调参指南

- 模型大小/位置: 修改 Live2DRenderrer.cs 顶部的 LIVE2D_SCALE (默认 56.25) 和 LIVE2D_OFFSET_Y (默认 0)
- 动作时长: 各动作对应的 *_DURATION 常量
- 权重调整: 通过右键菜单 UI 实时调整
- 过渡平滑度: WALK_FADE_IN_DURATION (0.3s)、IDLE_BLEND_DURATION (0.4s)
- 物理: DesktopPet.cs 顶部的重力/速度常量
- 输入栏位置: BottomInputBar.cs 顶部的 BAR_LEFT/RIGHT/TOP/BOTTOM 常量

17.3 构建注意

- Camera Background 必须为纯黑 (0,0,0,0) (DWM 玻璃层透明用)
- 构建版自动应用透明窗口设置 (编辑器版不执行)

- Live2D 模型需通过 Cubism SDK 导入生成 Prefab
- 单例互斥锁防止 Build & Run 产生多个实例
- 必须在 Project Settings 中：Graphics API = D3D11，取消勾选 DXGI flip model

18 待改进项

18.1 AI 活动感知与性格优化

当前 AI 活动感知和性格表达存在以下改进空间：

表 12: AI 活动感知改进方向

维度	现状	改进目标
活动感知粒度	8 个 笼 统 分 类 （coding/gaming/studying/browsing 等） ， GetForegroundWindow 快照模式	窗口标题已实时注入 System-Prompt, DeepSeek 可自行理解用户活动
多窗口感知	仅 追 踪 最 顶 层 窗 口， EnumWindows 未使用	EnumWindows 枚举所有可见窗口，每 10s 刷新多窗口摘要注入 SystemPrompt
浏览器深度	仅识别进程名	Windows UI Automation 读取浏览器标签页标题，零依赖零安装
性格温婉约束	SystemPrompt.txt 缺少”不要评判/说教”约束	加入温婉指令抑制 AI 说教倾向（ 已修复：System-Prompt.txt 新增「温婉守则」）
傲娇权重	IdleChatGenerator 的 system prompt 中”傲娇”权重过高	平衡”傲”与”娇”的比例（ 已修复：3:7 傲: 娇）
窗口标题利用	Classify() 后丢弃标题	传标题给 AI 用于回复上下文（ 已修复：ActivityTracker + ChatManager 实时注入）
API Key 配置	ChatManager / TimeWeather-Controller 各维护自己的 Key 字符串	统一到 ChatConfig 环境变量（ 已修复：配置中心化 + PetConfig 去 Key 持久化）
代码重复	ChatManager / IdleChat-Generator 各 维 护 一 套 HTTP/JSON 工具	提取到 ApiClient 共享静态类（ 已修复：两个调用者均重构使用 ApiClient）

参考开源方案： [ActivityWatch](#) (GitHub 14k+ stars) 是最大的开源时间追踪项目，但其 Windows 实现同样使用 `GetForegroundWindow` 单窗口追踪。改进方向可借鉴其生态中的浏览器扩展 `aw-watcher-web` 获取标签页 URL，以及 `aw-watcher-afk` 组件检测空闲状态。

19 版本历史

表 13: 版本历史

日期	版本	变更
2025	v0.1	透明窗口 + 基础物理 + Live2D 加载
2025	v0.2	地面状态机 + 走路动画
2025	v0.3	10 种空闲动作 + 强制动作系统
2025	v0.4	右键菜单 UI + 权重编辑
2025	v0.5	动作锁定机制 + 走路淡入过渡
2026-06-13	v0.6	本文档 + 项目文档整理
2026-06-17	v0.7	修复行走手臂被物理覆盖问题：添加 [DefaultExecutionOrder(801)] 确保 LateUpdate 跑在 CubismPhysicsController 之后； 体态参数提前到 Update() 设置供物理驱动衣服； 分离大/小范围手臂参数常量避免 clamp 满幅； 走路时 ForceUpdateNow 强制网格重算
2026-06-17	v0.8	头发/裙子/法盘直接物理驱动（20+ 输出参数 绕过物理延迟） 分区点击反馈（头/身/腿 3 区 + 参数保存防覆 盖） 拖拽挣扎动画（双臂划水 + 双腿交替 + 扭动 + 表情） 时间/天气响应系统（wttr.in API + 昼夜犯困 + 天气表情） 新增第 11 个空闲动作「困惑」 待机气泡支持时间/天气特化内容 更新项目文档 README + LaTeX
2026-06-18	v0.9	窗口透明从色键（#00FF00）切换为 DWM 玻 璃层透明（DwmExtendFrameIntoClientArea） 移除 Live2D 星辉/法阵/数钱全部视觉特效参数 （Param121/132/137 等） 简化聊天 UI：移除消息历史/状态栏/工具栏， 仅保留输入框 + 发送按钮 底部输入栏（BottomInputBar）改为 Windows 搜索风格 + 固定坐标系统 输入栏坐标可调 （BAR_LEFT/RIGHT/TOP/BOTTOM 常量） 输入框放大 + 淡入动画 Camera 背景统一改为纯黑 (0,0,0,0) Model3DRenderer 背景同样改为黑色 重建 BuildScript.cs（自动构建脚本）